Markov Chains

Geometric Algorithms

CAS CS 132

Objectives

Finish our discussion of LU factorization

Motivate linear dynamical systems

Analyze Markov chains and their properties

Learn to solve for steady-states

Keywords

linear dynamical systems

recurrence relations

linear difference equations

state vector

probability vector

stochastic matrix

Markov chain

steady-state vector

random walk

state diagram

Recap: LU Factorization

Recall: LU Factorization

Given $A \in \mathbb{R}^{m \times n}$, we'll write A as

echelon form of A

$$A = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ * & 1 & 0 & 0 \\ * & * & 1 & 0 \\ * & * & * & 1 \end{bmatrix}}_{L} \underbrace{\begin{bmatrix} \blacksquare & * & * & * & * \\ 0 & \blacksquare & * & * & * \\ 0 & 0 & 0 & \blacksquare & * \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}}_{U}$$

Gaussian Elimination and Elementary Matrices

$$A \sim E_1 A \sim E_2 E_1 A \sim \ldots \sim E_k E_{k-1} \ldots E_2 E_1 A$$

Gaussian Elimination and Elementary Matrices

$$A \sim E_1 A \sim E_2 E_1 A \sim \ldots \sim E_k E_{k-1} \ldots E_2 E_1 A$$

If B is the echelon form of A we get then

$$B = (E_k E_{k-1} \ldots E_2 E_1) A = EA$$

where E implements a *sequence* of row operations

Gaussian Elimination and Elementary Matrices

$$A \sim E_1 A \sim E_2 E_1 A \sim \ldots \sim E_k E_{k-1} \ldots E_2 E_1 A$$

If B is the echelon form of A we get then

Invertible

$$B = (E_k E_{k-1} \ldots E_2 E_1) A = EA$$

where E implements a *sequence* of row operations

Gaussian Elimination and Elementary Matrices

$$A \sim E_1 A \sim E_2 E_1 A \sim \ldots \sim E_k E_{k-1} \ldots E_2 E_1 A$$

If B is the echelon form of A we get then

Invertible

$$B = (E_k E_{k-1} \ldots E_2 E_1) A = EA$$

where E implements a *sequence* of row operations

$$A = E^{-1} B = (E_1^{-1} E_2^{-1} \ldots E_{k-1}^{-1} E_k^{-1}) B$$

LU Factorization Algorithm

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
3      U ← A
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
3      U ← A
4      convert U to an echelon form by GE forward step # without swaps
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
3      U ← A
4      convert U to an echelon form by GE forward step # without swaps
5      FOR each row operation OP in the prev step:
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
3      U ← A
4      convert U to an echelon form by GE forward step # without swaps
5      FOR each row operation OP in the prev step:
6          E ← the matrix implementing OP
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
3      U ← A
4      convert U to an echelon form by GE forward step # without swaps
5      FOR each row operation OP in the prev step:
6          E ← the matrix implementing OP
7          L ← L @ E^-1    # note the multiplication on the right
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
3      U ← A
4      convert U to an echelon form by GE forward step # without swaps
5      FOR each row operation OP in the prev step:
6          E ← the matrix implementing OP
7          L ← L @ E^-1    # note the multiplication on the right
8      RETURN (L, U)
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
3      U ← A
4      convert U to an echelon form by GE forward step # without swaps
5      FOR each row operation OP in the prev step:
6          E ← the matrix implementing OP
7          L ← L @ E^-1    # note the multiplication on the right
8      RETURN (L, U)
```

we can do this more efficiently

The forward part of Gaussian elimination is a matrix factorization

demo

How To: LU Factorization by hand

Q. Find a LU Factorization for the matrix A (assuming no swaps)

A.

» Start with L as the identity matrix
» Find U by the forward part of GE
» For each operation $R_i \leftarrow R_i + kR_j$, set L_{ij} to $-k$

Example

$$\begin{bmatrix} 1 & 0 & 3 \\ 2 & -1 & 7 \\ 1 & 3 & 2 \end{bmatrix}$$

Analyzing LU Factorization

Analyzing Linear Algebra Algorithms

Analyzing Linear Algebra Algorithms

We will not use $O(\cdot)$ notation

Analyzing Linear Algebra Algorithms

We will not use $O(\cdot)$ notation

We care about **FL**oating-oint **OP**erations (FLOPs):

- addition
- subtraction
- multiplication
- division
- square root

Analyzing Linear Algebra Algorithms

We will not use $O(\cdot)$ notation

We care about **FL**oating-oint **OP**erations (FLOPs):

- >> addition
- >> subtraction
- >> multiplication
- >> division
- >> square root

$2n$ vs. n is very different when $n \sim 10^{20}$

Dominant Terms

Dominant Terms

$f(n)$ is ***asymptotically equivalent*** to $g(n)$ if

$$\lim_{i \to \infty} \frac{f(i)}{g(i)} = 1$$

Dominant Terms

$f(n)$ is ***asymptotically equivalent*** to $g(n)$ if

$$\lim_{i \to \infty} \frac{f(i)}{g(i)} = 1$$

For polynomials, they're equivalent to their dominant term

Dominant Terms

The dominant term of a polynomial is the monomial with the highest degree

$$\lim_{i \to \infty} \frac{3x^3 + 100000x^2}{3x^3} = 1$$

$3x^3$ dominates the function even though the coefficient for x^2 is so large

How To: Solving systems with the LU

Q. Solve the equation $A\mathbf{x} = \mathbf{b}$ given that $A = LU$ is a LU factorization

A. First solve $L\mathbf{x} = \mathbf{b}$ to get a solution $\mathbf{c}$, then solve $U\mathbf{x} = \mathbf{c}$ to get a solution $\mathbf{d}$

How To: Solving systems with the LU

Q. Solve the equation $A\mathbf{x} = \mathbf{b}$ given that $A = LU$ is a LU factorization

A. First solve $L\mathbf{x} = \mathbf{b}$ to get a solution $\mathbf{c}$, then solve $U\mathbf{x} = \mathbf{c}$ to get a solution $\mathbf{d}$

Why is this better than just solving $A\mathbf{x} = \mathbf{b}$***?***

FLOPs for Solving General Systems

The following FLOP estimates are based on $n \times n$ matrices

FLOPs for Solving General Systems

Gaussian Elimination: $\sim \frac{2n^3}{3}$ FLOPS

The following FLOP estimates are based on $n \times n$ matrices

FLOPs for Solving General Systems

Gaussian Elimination: $\sim \frac{2n^3}{3}$ FLOPS

GE Forward: $\sim \frac{2n^3}{3}$ FLOPS

The following FLOP estimates are based on $n \times n$ matrices

FLOPs for Solving General Systems

Gaussian Elimination: $\sim \frac{2n^3}{3}$ FLOPS

GE Forward: $\sim \frac{2n^3}{3}$ FLOPS

GE Backward: $\sim 2n^2$ FLOPS

The following FLOP estimates are based on $n \times n$ matrices

FLOPs for Solving General Systems

Gaussian Elimination: $\sim \frac{2n^3}{3}$ FLOPS

GE Forward: $\sim \frac{2n^3}{3}$ FLOPS

GE Backward: $\sim 2n^2$ FLOPS

Matrix Inversion: $\sim 2n^3$ FLOPS

The following FLOP estimates are based on $n \times n$ matrices

FLOPs for Solving General Systems

Gaussian Elimination: $\sim \frac{2n^3}{3}$ FLOPS

GE Forward: $\sim \frac{2n^3}{3}$ FLOPS

GE Backward: $\sim 2n^2$ FLOPS

Matrix Inversion: $\sim 2n^3$ FLOPS

Matrix–Vector Multiplication: $\sim 2n^2$ FLOPS

The following FLOP estimates are based on $n \times n$ matrices

FLOPs for Solving General Systems

Gaussian Elimination: $\sim \frac{2n^3}{3}$ FLOPS

GE Forward: $\sim \frac{2n^3}{3}$ FLOPS

GE Backward: $\sim 2n^2$ FLOPS

Matrix Inversion: $\sim 2n^3$ FLOPS

Matrix–Vector Multiplication: $\sim 2n^2$ FLOPS

Solving by matrix inversion: $\sim 2n^3$ FLOPS

The following FLOP estimates are based on $n \times n$ matrices

FLOPs for Solving General Systems

Gaussian Elimination: $\sim \frac{2n^3}{3}$ FLOPS

GE Forward: $\sim \frac{2n^3}{3}$ FLOPS

GE Backward: $\sim 2n^2$ FLOPS

Matrix Inversion: $\sim 2n^3$ FLOPS

Matrix–Vector Multiplication: $\sim 2n^2$ FLOPS

Solving by matrix inversion: $\sim 2n^3$ FLOPS

Solving by Gaussian elimination: $\sim \frac{2n^3}{3}$ FLOPS

The following FLOP estimates are based on $n \times n$ matrices

FLOPS for solving LU systems

FLOPS for solving LU systems

LU Factorization: $\sim \frac{2n^3}{3}$ FLOPS

FLOPS for solving LU systems

LU Factorization: $\sim \frac{2n^3}{3}$ FLOPS

Solving $L\mathbf{x} = \mathbf{b}$: $\sim 2n^2$ FLOPS (by "forward" substitution)

FLOPS for solving LU systems

LU Factorization: $\sim \frac{2n^3}{3}$ FLOPS

Solving $L\mathbf{x} = \mathbf{b}$: $\sim 2n^2$ FLOPS (by "forward" substitution)

Solving $U\mathbf{x} = \mathbf{c}$: $\sim 2n^2$ FLOPS (by back substitution)

FLOPS for solving LU systems

LU Factorization: $\sim \frac{2n^3}{3}$ FLOPS

Solving $L\mathbf{x} = \mathbf{b}$: $\sim 2n^2$ FLOPS (by "forward" substitution)

Solving $U\mathbf{x} = \mathbf{c}$: $\sim 2n^2$ FLOPS (by back substitution)

Solving by LU Factorization: $\sim \frac{2n^3}{3}$ FLOPS

If you solve several matrix equations for the same matrix, LU factorization is faster than matrix inversion on the first equation, and the same (asymptotically) in later equations (and it works for rectangular matrices).

Other Considerations: Density

Other Considerations: Density

If A doesn't have to many nonzero entries (A is **sparse)** then its likely that L and U won't either.

Other Considerations: Density

If A doesn't have to many nonzero entries (A is **sparse)** then its likely that L and U won't either.

But A^{-1} may have ***many*** entries (A^{-1} is **dense**)

Other Considerations: Density

If A doesn't have to many nonzero entries (A is **sparse)** then its likely that L and U won't either**.**

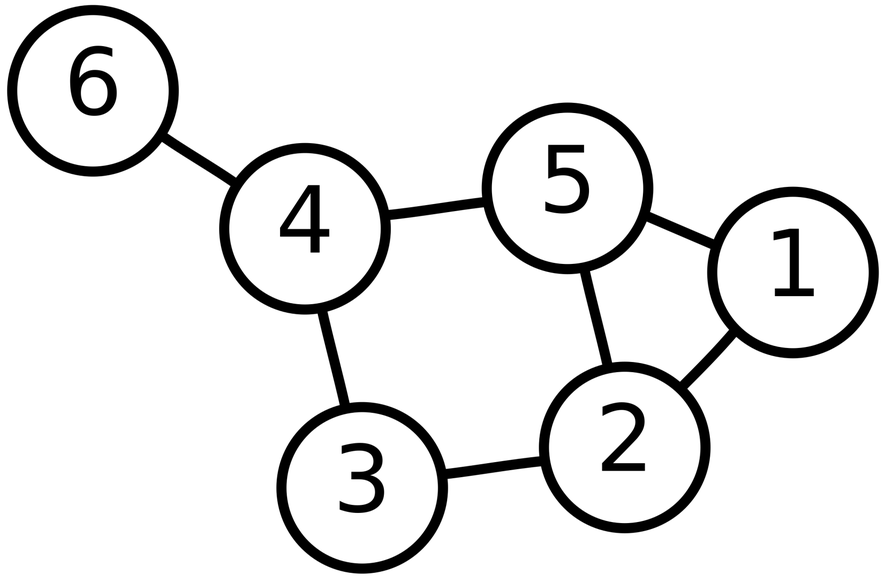
But A^{-1} may have ***many*** entries (A^{-1} is **dense**)

Sparse matrices are faster to compute with and better with respect to storage

Graph Theory Primer

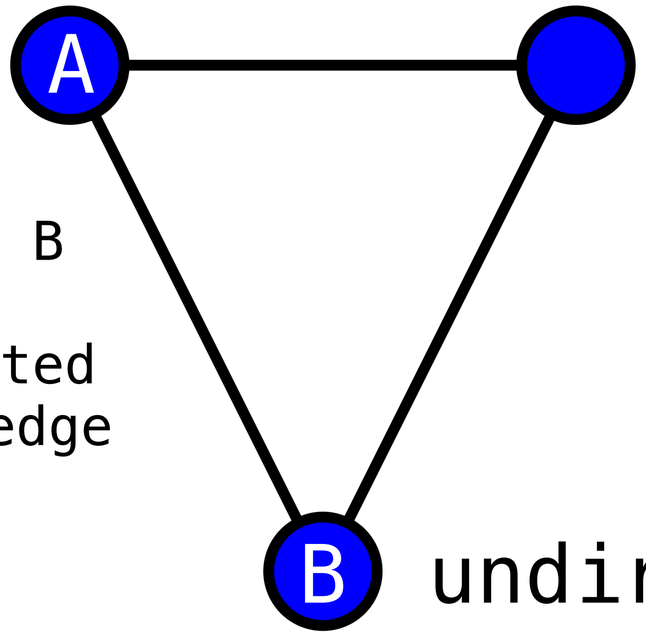
Graphs

Def (Informal). A **graph** is a collection of nodes with edges between them



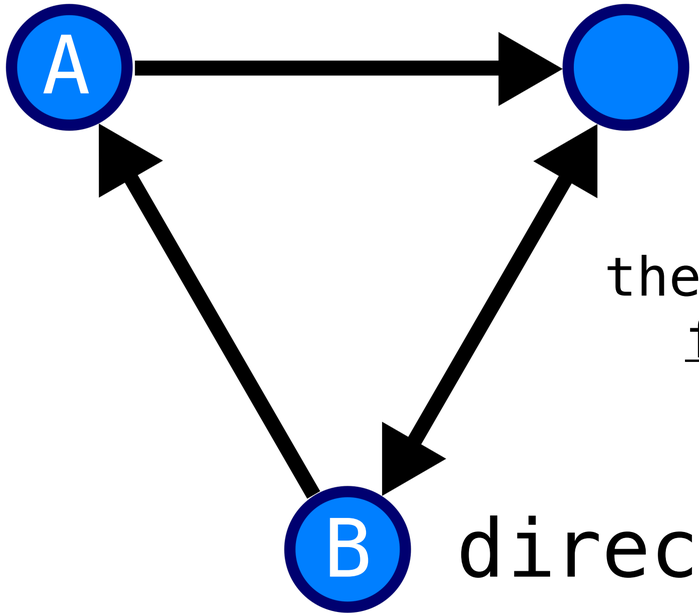
Directed vs. Undirected Graphs

A graph is **directed** if its edges have a direction



A and B are connected by an edge

undirected

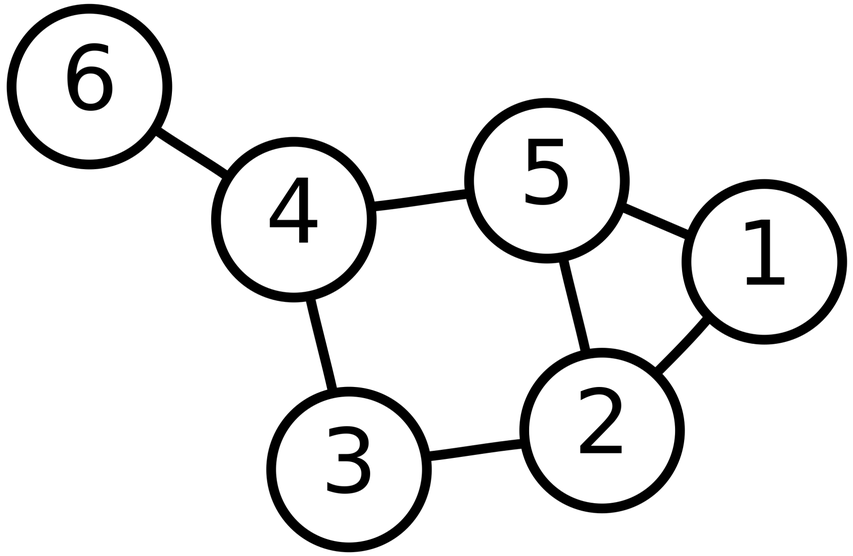


there is an edge from B to A

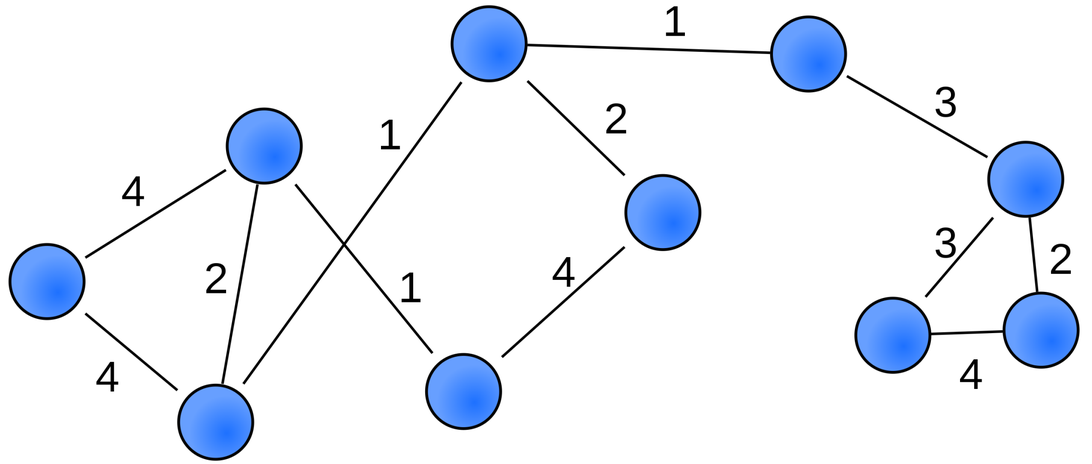
directed

Weighted vs Unweighted graphs

A graph is **weighted** if its edges have associated values



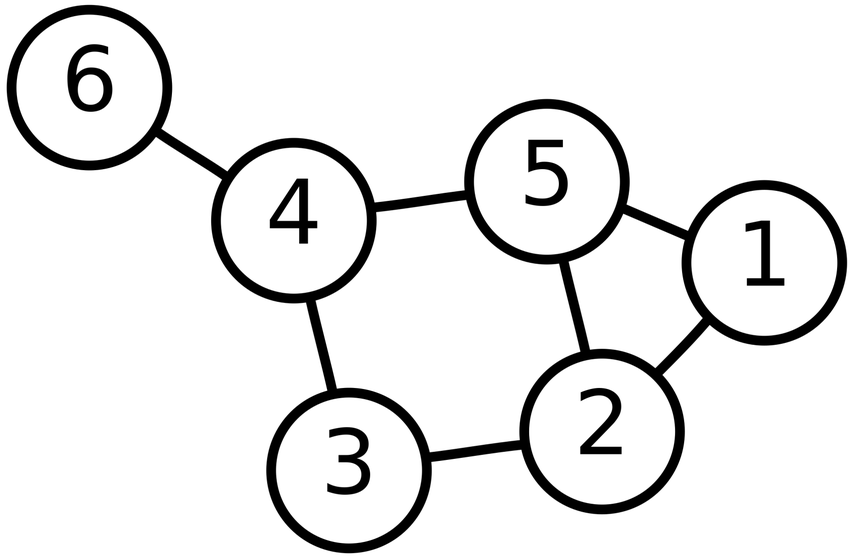
unweighted



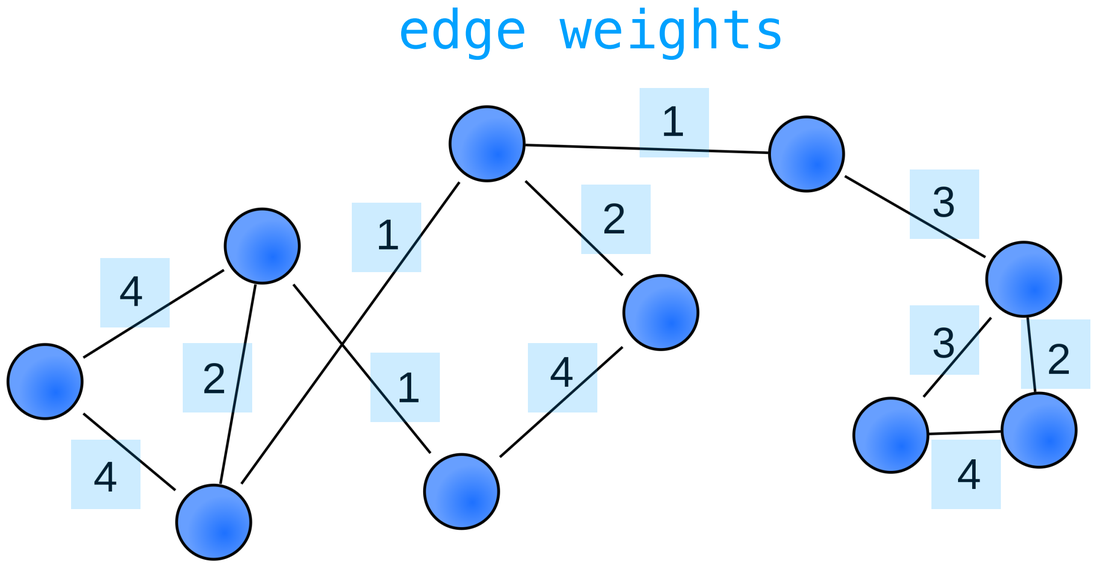
weighted

Weighted vs Unweighted graphs

A graph is **weighted** if its edges have associated values



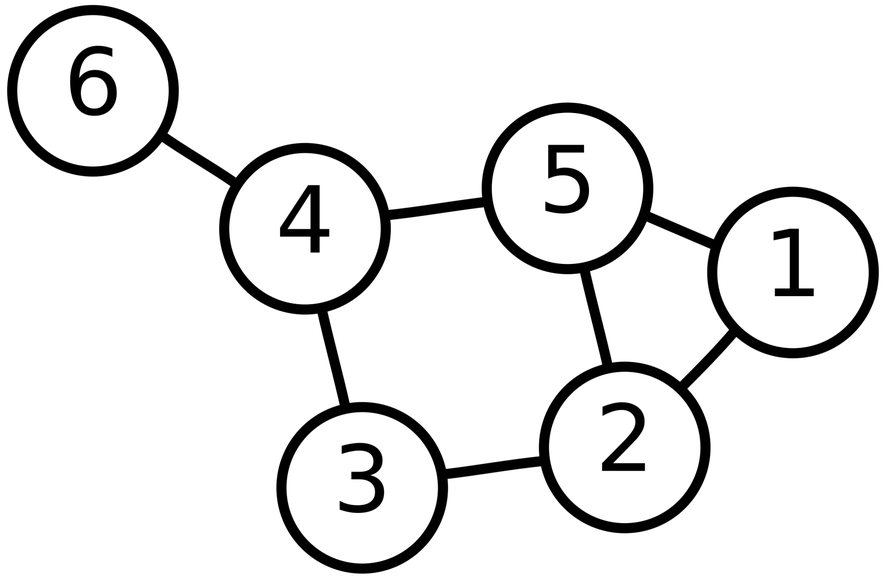
unweighted



weighted

Simple Graphs

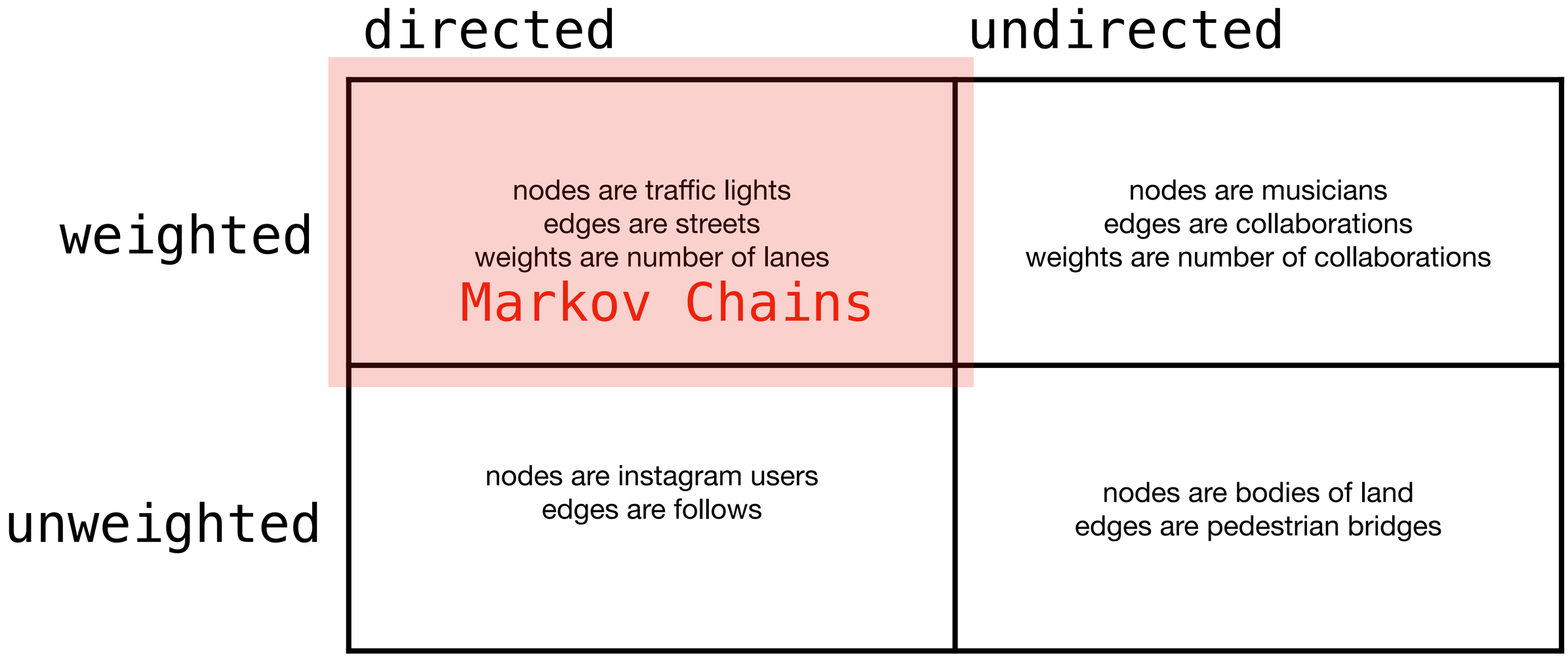
A graph is **simple** if it is undirected, has no self loops, and no multi-edges



Four Kinds of Graphs

	directed	undirected
weighted	nodes are traffic lights edges are streets weights are number of lanes	nodes are musicians edges are collaborations weights are number of collaborations
unweighted	nodes are instagram users edges are follows	nodes are bodies of land edges are pedestrian bridges

Four Kinds of Graphs



Fundamental Question

Fundamental Question

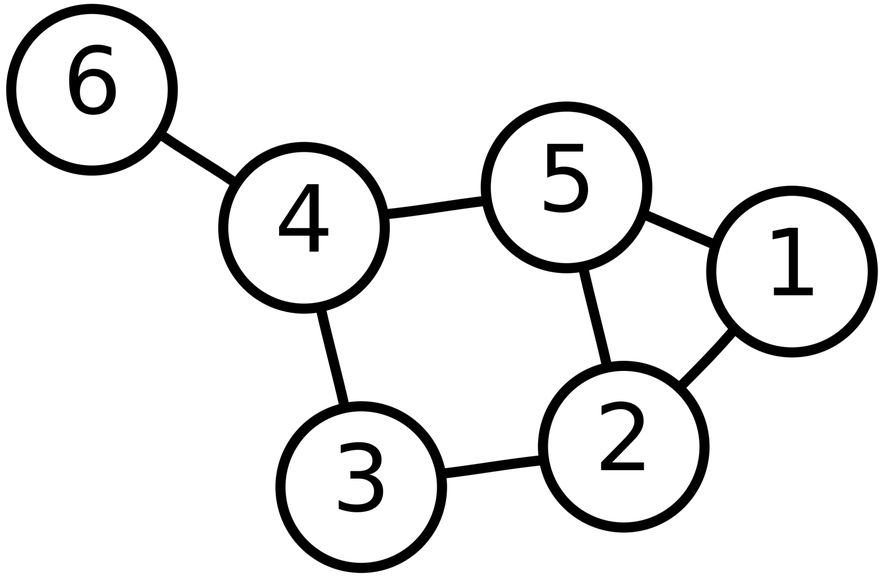
How do we represent a graph formally in a computer?

Fundamental Question

How do we represent a graph formally in a computer?

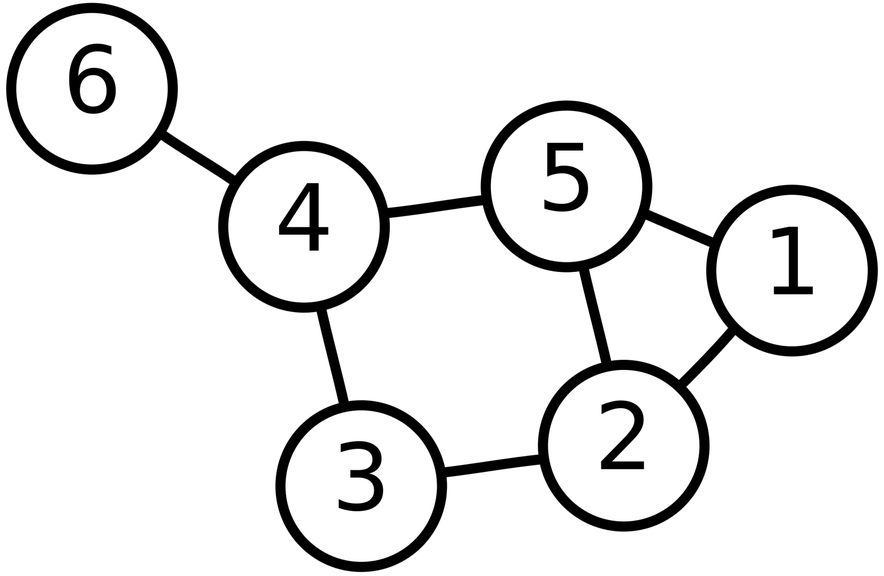
There are a couple ways, but one way is to use **matrices**

Adjacency Matrices



Adjacency Matrices

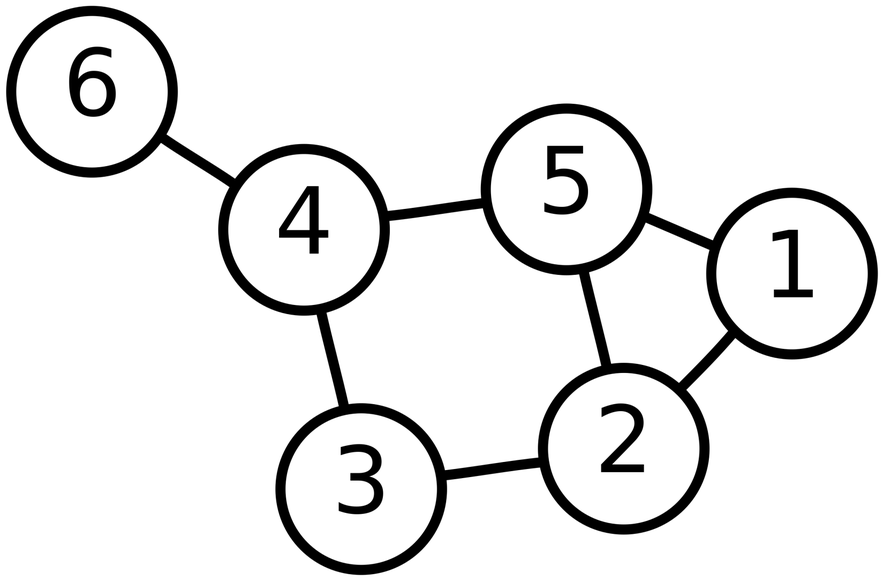
Let G be an simple graph with its nodes labeled by numbers 1 through n



Adjacency Matrices

Let G be an simple graph with its nodes labeled by numbers 1 through n

We can create the **adjacency matrix** A for G as:



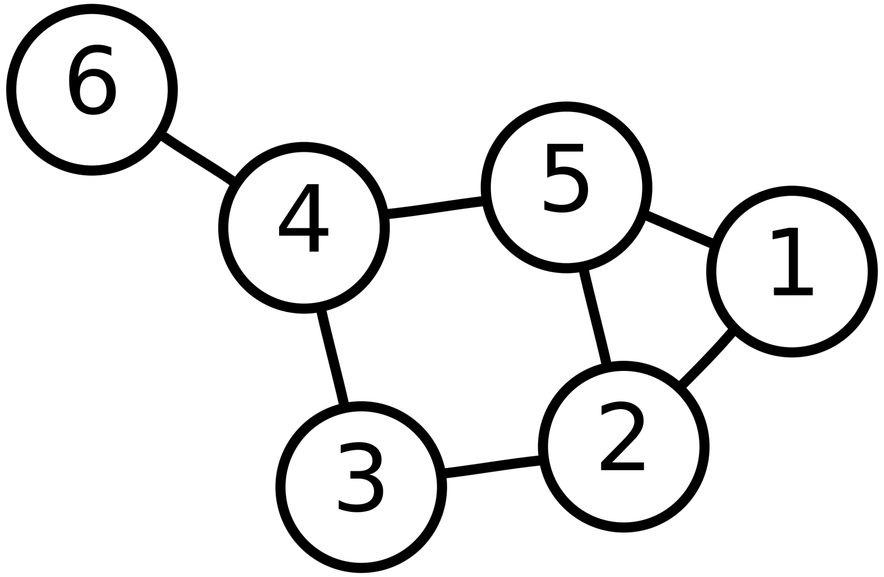
Adjacency Matrices

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Let G be an simple graph with its nodes labeled by numbers 1 through n

We can create the **adjacency matrix** A for G as:

$$A_{ij} = \begin{cases} 1 & \text{edge between i and j} \\ 0 & \text{otherwise} \end{cases}$$



Once we have adjacency matrices we can do linear algebra on graphs

Dynamical Systems

Change

Change

Things change

Change

Things change

Things change from one state of affairs to another state of affairs

Change

Things change

Things change from one state of affairs to another state of affairs

Things change often in unpredictable ways

Change

Things change

Things change from one state of affairs to another state of affairs

Things change often in unpredictable ways

If something changes unpredictably, what can we say about it?

Dynamical Systems

Dynamical Systems

Def (Informal). A **dynamical system** is a thing (with interacting parts) that changes over time

Dynamical Systems

Def (Informal). A **dynamical system** is a thing (with interacting parts) that changes over time

A dynamical system *transitions* between *possible states* over time

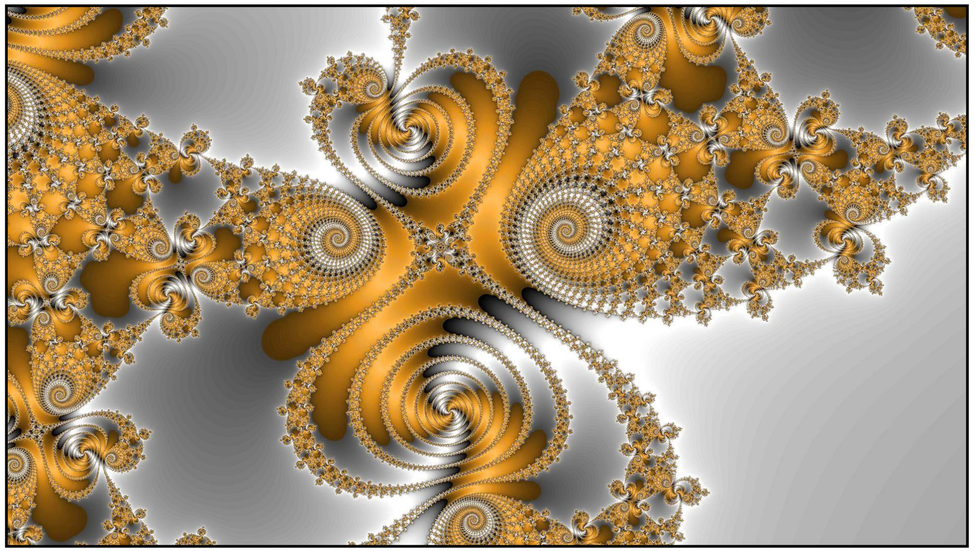
Dynamical Systems

Def (Informal). A **dynamical system** is a thing (with interacting parts) that changes over time

A dynamical system *transitions* between *possible states* over time

» economics (stocks)
» physical/chemical systems
» populations
» weather

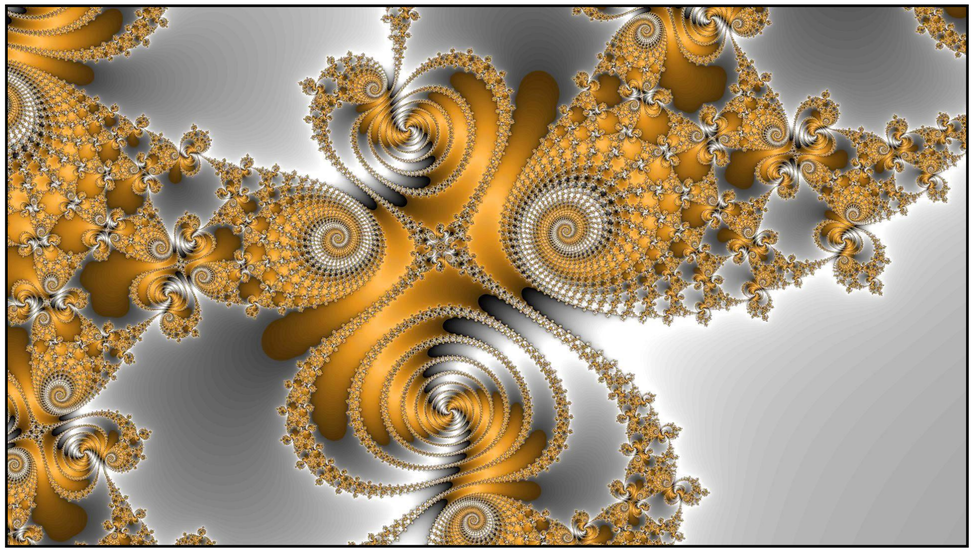
An Aside: Chaos Theory



https://commons.wikimedia.org/wiki/File:Fr137.jpg

An Aside: Chaos Theory

Complex systems like the weather
or the economy *look* random

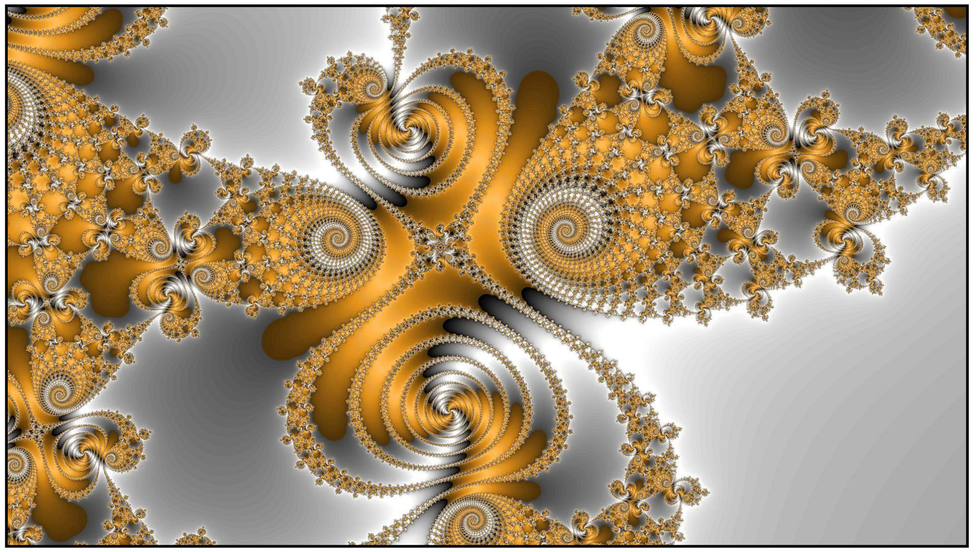


https://commons.wikimedia.org/wiki/File:Fr137.jpg

An Aside: Chaos Theory

Complex systems like the weather
or the economy *look* random

But even in chaotic systems
there are *underlying patterns*
and *repeated structures*



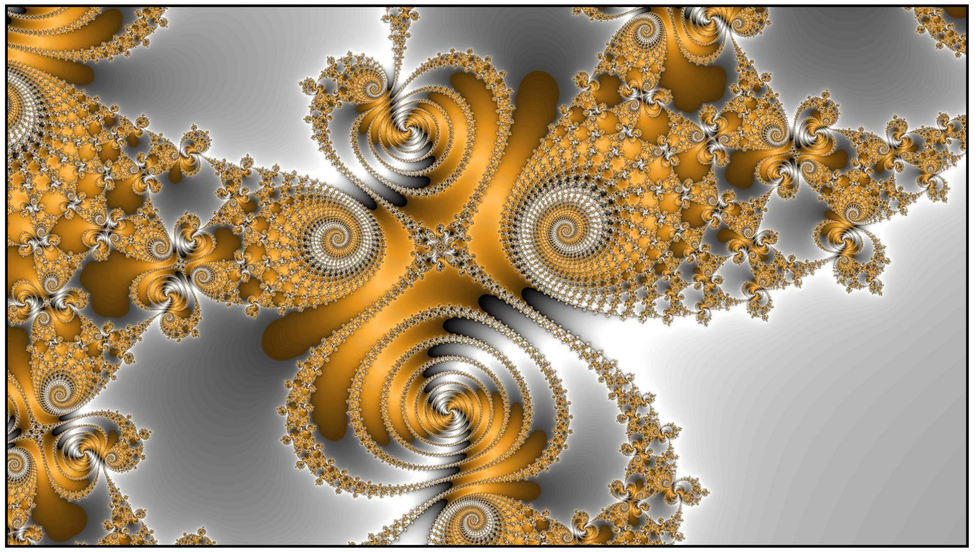
https://commons.wikimedia.org/wiki/File:Fr137.jpg

An Aside: Chaos Theory

Complex systems like the weather or the economy *look* random

But even in chaotic systems there are *underlying patterns* and *repeated structures*

Often it's useful to consider chaotic systems in terms of *global* properties



https://commons.wikimedia.org/wiki/File:Fr137.jpg

Motivating Questions

Motivating Questions

What does a system look like "in the long term"?

Motivating Questions

What does a system look like "in the long term"?

Does it reach a kind of *equilibrium*?

Motivating Questions

What does a system look like "in the long term"?

Does it reach a kind of *equilibrium*?

Does a part of the system *dominate* over time?

Linear Dynamical Systems

Linear Dynamical Systems

Linear Dynamical Systems

Def. A **(discrete time) linear dynamical system** is a described a $n \times n$ matrix A**.** It's **evolution function** is the matrix transformation $\mathbf{x} \mapsto A\mathbf{x}$

Linear Dynamical Systems

Def. A **(discrete time) linear dynamical system** is a described a $n \times n$ matrix A. It's **evolution function** is the matrix transformation $\mathbf{x} \mapsto A\mathbf{x}$

The **possible states** of the system are vectors in $\mathbb{R}^n$

Linear Dynamical Systems

Def. A **(discrete time) linear dynamical system** is a described a $n \times n$ matrix A. It's **evolution function** is the matrix transformation $\mathbf{x} \mapsto A\mathbf{x}$

The **possible states** of the system are vectors in $\mathbb{R}^n$

Given an **initial state vector** $\mathbf{v}_0$, we can determine the **state vector** of the system after i time steps:

$$\mathbf{v}_i = A\mathbf{v}_{i-1}$$ *(linear recurrence relation)*

State Vectors

$$
\begin{aligned}
&\mathbf{v}_0 \\
&\mathbf{v}_1 = A\mathbf{v}_0 \\
&\mathbf{v}_2 = A\mathbf{v}_1 = A(A\mathbf{v}_0) \\
&\mathbf{v}_3 = A\mathbf{v}_2 = A(AA\mathbf{v}_0) \\
&\mathbf{v}_4 = A\mathbf{v}_3 = A(AAA\mathbf{v}_0) \\
&\mathbf{v}_5 = A\mathbf{v}_4 = A(AAAA\mathbf{v}_0) \\
&\quad\vdots
\end{aligned}
$$

The **state vector** $\mathbf{v}_k$ is the state of the system after k time steps

Warm up:
Population Dynamics

The Setup

The Setup

We're working for the census. We have population measurements for a city and a suburb which are geographically coincident

The Setup

We're working for the census. We have population measurements for a city and a suburb which are geographically coincident

We find by analyzing previous data that each year:

- 5% of the population moves from city → suburb
- 3% of the population moves from suburb → city

Fundamental Question

Can we make any predictions about the population of the city and suburb in 20 years?

Assumptions: No immigration, emigration, birth, death, etc. **The overall population stays fixed.**

Setting up Linear Equations

Setting up Linear Equations

If city_0 = city pop. = 600,000

and suburb_0 = suburb pop. = 400,000

Setting up Linear Equations

If city_0 = city pop. = 600,000
and suburb_0 = suburb pop. = 400,000

then the populations next year are given by:

$$\text{city}_1 = (0.95)\text{city}_0 + (0.03)\text{suburb}_0$$

$$\text{suburb}_1 = (0.05)\text{city}_0 + (0.97)\text{suburb}_0$$

Setting up Linear Equations

If city_0 = city pop. = 600,000
and suburb_0 = suburb pop. = 400,000

then the populations next year are given by:

$$\text{city}_1 = (0.95)\text{city}_0 + (0.03)\text{suburb}_0$$
$$\text{suburb}_1 = (0.05)\text{city}_0 + (0.97)\text{suburb}_0$$

people who stayed

people who left

Setting up a Matrix

$$\begin{bmatrix} \text{city}_1 \\ \text{suburb}_1 \end{bmatrix} = \begin{bmatrix} 0.95 & 0.3 \\ 0.05 & 0.97 \end{bmatrix} \begin{bmatrix} \text{city}_0 \\ \text{suburb}_0 \end{bmatrix} = \begin{bmatrix} 582{,}000 \\ 418{,}000 \end{bmatrix}$$

We expect the population of the city to decrease in a year

Setting up a Matrix

$$\begin{bmatrix} \text{city}_2 \\ \text{suburb}_2 \end{bmatrix} = \begin{bmatrix} 0.95 & 0.3 \\ 0.05 & 0.97 \end{bmatrix} \begin{bmatrix} \text{city}_1 \\ \text{suburb}_1 \end{bmatrix} = \begin{bmatrix} 565{,}440 \\ 434{,}560 \end{bmatrix}$$

The next year, we expect the population of the city to *continue* to decrease

Will it decrease indefinitely?

Setting up a Matrix

$$\begin{bmatrix} \text{city}_k \\ \text{suburb}_k \end{bmatrix} = \begin{bmatrix} 0.95 & 0.3 \\ 0.05 & 0.97 \end{bmatrix} \begin{bmatrix} \text{city}_{k-1} \\ \text{suburb}_{k-1} \end{bmatrix}$$

Setting up a Matrix

$$\begin{bmatrix} \text{city}_k \\ \text{suburb}_k \end{bmatrix} = \begin{bmatrix} 0.95 & 0.3 \\ 0.05 & 0.97 \end{bmatrix} \begin{bmatrix} \text{city}_{k-1} \\ \text{suburb}_{k-1} \end{bmatrix}$$

This is a *linear dynamical system*

Setting up a Matrix

$$\begin{bmatrix} \text{city}_k \\ \text{suburb}_k \end{bmatrix} = \begin{bmatrix} 0.95 & 0.3 \\ 0.05 & 0.97 \end{bmatrix} \begin{bmatrix} \text{city}_{k-1} \\ \text{suburb}_{k-1} \end{bmatrix}$$

This is a *linear dynamical system*

To guess what the population will look like in 20 years, we need to compute

$$\begin{bmatrix} 0.95 & 0.03 \\ 0.05 & 0.97 \end{bmatrix}^{20} \begin{bmatrix} \text{city}_0 \\ \text{suburb}_0 \end{bmatrix}$$

demo

Markov Chains

Stochastic Matrices

$$\begin{bmatrix} 0.95 & 0.03 \\ 0.05 & 0.97 \end{bmatrix}$$

Stochastic Matrices

$$\begin{bmatrix} 0.95 & 0.03 \\ 0.05 & 0.97 \end{bmatrix}$$

What's special about this matrix?

Stochastic Matrices

$$\begin{bmatrix} 0.95 & 0.03 \\ 0.05 & 0.97 \end{bmatrix}$$

What's special about this matrix?

» Its entries are nonnegative

Stochastic Matrices

$$\begin{bmatrix} 0.95 & 0.03 \\ 0.05 & 0.97 \end{bmatrix}$$

What's special about this matrix?

- Its entries are nonnegative
- Its columns sum to 1

Stochastic Matrices

$$\begin{bmatrix} 0.95 & 0.03 \\ 0.05 & 0.97 \end{bmatrix}$$

What's special about this matrix?

- Its entries are nonnegative
- Its columns sum to 1

This should make us think probability

Stochastic Matrices

Def. A matrix in $\mathbb{R}^{n \times n}$ is **stochastic** if its entries are nonnegative and its columns sum to 1

$$\begin{bmatrix} 0.7 & 0.1 & 0.3 \\ 0.2 & 0.8 & 0.3 \\ 0.1 & 0.1 & 0.4 \end{bmatrix}$$

Markov Chains

Def. A **Markov chain** is a linear dynamical system whose evolution function is given by a *stochastic* matrix

State Diagrams

Def. A **state diagram** is a directed weighted graph whose adjacency matrix is stochastic.

Ex.

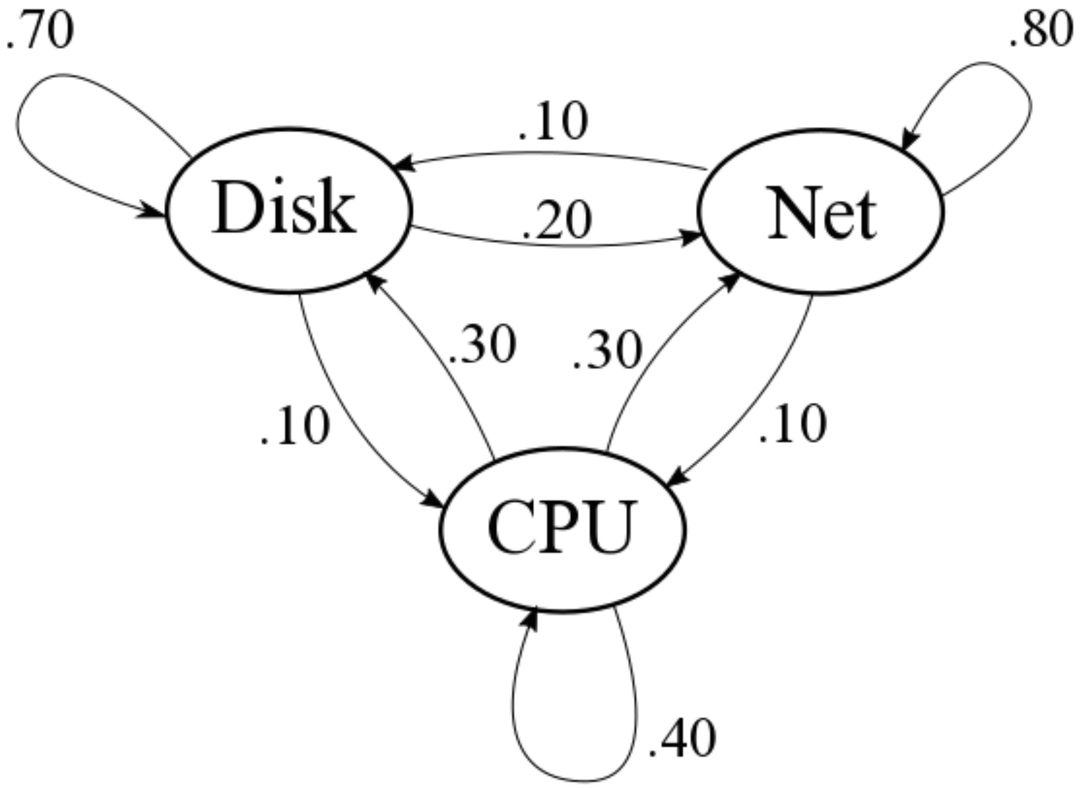


Another Example: Computer System

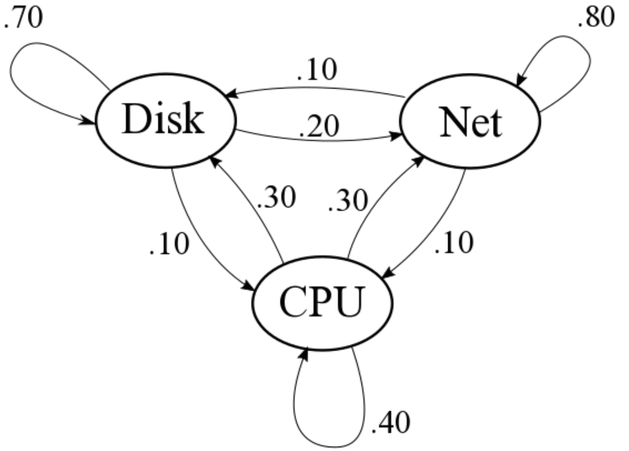
Imagine a computer system in which tasks request service from disk, network or CPU

In the long term, which device is busiest?

This is about finding a stable state



Example



Key Property of Stochastic Matrices

Key Property of Stochastic Matrices

Stochastic matrices *redistribute* the "stuff" in a vector

Key Property of Stochastic Matrices

Stochastic matrices *redistribute* the "stuff" in a vector

Thm. For a stochastic matrix A and a vector $\mathbf{v}$,

sum of entries of $\mathbf{v}$ = sum of entries of $A\mathbf{v}$

Key Property of Stochastic Matrices

Thm. For stochastic A and a vector $\mathbf{v}$

$$\mathbf{1}^T\mathbf{v} = \mathbf{1}^T(A\mathbf{v})$$

Returning to the Example

$$\begin{bmatrix} \text{city}_k \\ \text{suburb}_k \end{bmatrix} = \begin{bmatrix} 0.95 & 0.3 \\ 0.05 & 0.97 \end{bmatrix} \begin{bmatrix} \text{city}_{k-1} \\ \text{suburb}_{k-1} \end{bmatrix}$$

Returning to the Example

$$\begin{bmatrix} \text{city}_k \\ \text{suburb}_k \end{bmatrix} = \begin{bmatrix} 0.95 & 0.3 \\ 0.05 & 0.97 \end{bmatrix}^k \begin{bmatrix} \text{city}_0 \\ \text{suburb}_0 \end{bmatrix}$$

Returning to the Example

$$\begin{bmatrix} \text{city}_k \\ \text{suburb}_k \end{bmatrix} = \begin{bmatrix} 0.95 & 0.3 \\ 0.05 & 0.97 \end{bmatrix}^k \begin{bmatrix} 600{,}000 \\ 400{,}000 \end{bmatrix}$$

Returning to the Example

$$\begin{bmatrix} \text{city}_k \\ \text{suburb}_k \end{bmatrix} = \begin{bmatrix} 0.95 & 0.3 \\ 0.05 & 0.97 \end{bmatrix}^k \begin{bmatrix} 600{,}000 \\ 400{,}000 \end{bmatrix}$$

What if we want to analyze a different population?

Returning to the Example

$$\begin{bmatrix} \text{city}_k \\ \text{suburb}_k \end{bmatrix} = \begin{bmatrix} 0.95 & 0.3 \\ 0.05 & 0.97 \end{bmatrix}^k \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

60% of pop. in city

40% of pop. in suburb

We should think about **how the population is distributed** instead of a particular population

Probability Vectors

Probability Vectors

Def. A **probability vector** is a vector of nonnegative values that sum to 1. They represent:

Probability Vectors

Def. A **probability vector** is a vector of nonnegative values that sum to 1. They represent:

» discrete probability distributions

» distributions of collections of things

(these are really the same thing)

Probability Vectors (Example)

The vector $\begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$ represented the distribution of the population, but we can also think of this as:

If we choose a random person from the population we'll get someone:

in the city with probability 0.6

in the suburbs with probability 0.4

The Point

The Point

We'll be interested in the dynamics of Markov chains on **probability vectors**

The Point

We'll be interested in the dynamics of Markov chains on **probability vectors**

Since stochastic matrices preserve $\mathbf{1}^T\mathbf{v}$, they *transform* one distribution into another

The Point

We'll be interested in the dynamics of Markov chains on **probability vectors**

Since stochastic matrices preserve $\mathbf{1}^T\mathbf{v}$, they *transform* one distribution into another

Can we say something about how the distribution changes in the long run?

Steady-State Vectors

Steady-State Vectors

Def. A **steady-state vector** for a stochastic matrix A is a probability vector $\mathbf{q}$ such that

$$A\mathbf{q} = \mathbf{q}$$

A steady-state vector is *not changed* by the stochastic matrix. They describe *equilibriums*

Returning to the Example

Returning to the Example

How do we interpret a steady-state vector for our example?

Returning to the Example

How do we interpret a steady-state vector for our example?

The populations in the city and the suburb stay the same over time

Returning to the Example

How do we interpret a steady-state vector for our example?

The populations in the city and the suburb stay the same over time

The same number of people are moving into and out of the city each year

Fundamental Questions

Do steady states exist?

Are they unique?

How do we find them?

Finding Steady-State Vectors

$$A\mathbf{q} = \mathbf{q}$$

Let's solve this equation for $\mathbf{q}$:

How to: Steady-State Vectors

How to: Steady-State Vectors

Q. Determine if the Markov chain with stochastic matrix A has a steady-state vector. If it does, find such a vector

How to: Steady-State Vectors

Q. Determine if the Markov chain with stochastic matrix A has a steady-state vector. If it does, find such a vector

A. Solve the equation $(A - I)\mathbf{x} = \mathbf{0}$ and find a solution whose entries sum to 1 (this will be possible given a free variable)

How to: Steady-State Vectors

Q. Determine if the Markov chain with stochastic matrix A has a steady-state vector. If it does, find such a vector

A. Solve the equation $(A - I)\mathbf{x} = \mathbf{0}$ and find a solution whose entries sum to 1 (this will be possible given a free variable)

If there is no such solution, the system does not have a steady state

Example

$$\begin{bmatrix} 0.95 & 0.3 \\ 0.05 & 0.97 \end{bmatrix}$$

demo

Existence vs Convergence

If $(A - I)\mathbf{x} = \mathbf{0}$ has a nontrival solutions, then the system has a stable state. This **does not** mean:

- the stable state is unique
- the system *converges* to this state

Convergence

Convergence

Def. For a Markov chain with stochastic matrix A, an initial state $\mathbf{v}_0$ **converges** to the state $\mathbf{v}$ if $\lim_{k\to\infty} A^k\mathbf{v}_0 = \mathbf{v}$

Convergence

Def. For a Markov chain with stochastic matrix A, an initial state $\mathbf{v}_0$ **converges** to the state $\mathbf{v}$ if $\lim_{k\to\infty} A^k\mathbf{v}_0 = \mathbf{v}$

As we repeatedly multiply $\mathbf{v}_0$ by A, we get closer and closer to $\mathbf{v}$ (in the limit)

Example of Non-Convergence

Example of Non-Convergence

Non-Example. I is a stochastic matrix and

$$I\mathbf{v} = \mathbf{v}$$

for any choice of $\mathbf{v}$

Example of Non-Convergence

Non-Example. I is a stochastic matrix and

$$I\mathbf{v} = \mathbf{v}$$

for any choice of $\mathbf{v}$

So this system does not have a unique steady state

Example of Non-Convergence

Non-Example. I is a stochastic matrix and

$$I\mathbf{v} = \mathbf{v}$$

for any choice of $\mathbf{v}$

So this system does not have a unique steady state

And no vectors converge to the same stable state

Regular Stochastic Matrices

Regular Stochastic Matrices

Definition. A stochastic matrix A is **regular** if A^k has all positive entries for *some nonnegative* k

Regular Stochastic Matrices

Definition. A stochastic matrix A is **regular** if A^k has all positive entries for *some nonnegative* k

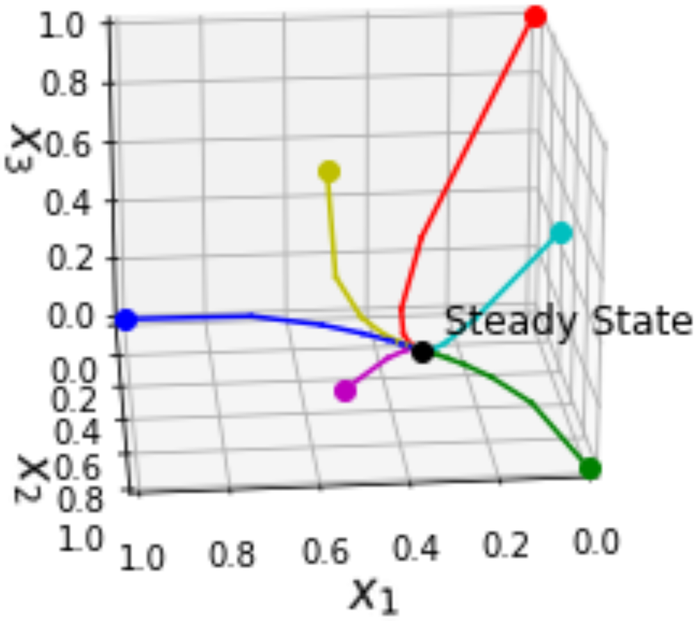
Theorem. A regular stochastic matrix P has a unique steady state, and

every probability vector converges to it

Mixing

This process of converging to a unique steady state is called "mixing"

This theorem says, after some amount of mixing, we'll be close to the stable state **no matter where we started**



Example

$$\begin{bmatrix} 0.5 & 0.4 & 0 \\ 0.5 & 0.4 & 0.5 \\ 0 & 0.2 & 0.5 \end{bmatrix}$$

Summary

Markov chains allow us to reason about dynamical systems that are dictated by some amount of randomness

Stable states represent equilibriums